Logarithmic Approximation for Covering Multi-Interface Networks

Michał Szyfelbein
Gdańsk University of Technology, Poland

Abstract

We give an $O(\log m)$ approximation algorithm for the min-sum and the min-max covering assignment problems in multi-interface networks, where m is the number of edges in the network. The algorithm is based on rounding the optimal solution of a natural, set-cover-style linear programming relaxation of the problem. We also show how to use this LP to obtain a k-approximation algorithm for both problems, where k is the number of interfaces. Importantly, all of the aforementioned procedures work even for the non uniform cost case, when the cost of activating an interface depends on the interface as well as the vertex to which it is assigned. This improves the previously best known approximation algorithms for the min-max version of the problem including: $O((1+b)\cdot(\ln\Delta + c_{\max}))$-approximation algorithm, where b is a certain parameter of the input instance which can be as large as $\Omega(n)$ and $c_{\max}$ is the maximum cost of activating an interface and $O(k)$-approximation algorithm for the uniform cost case. Moreover, we also give combinatorial $O(\Delta)$-approximation algorithms for both problems, thus improving on the previously known $O(\Delta \cdot c_{\max})$ for the min-max version of the problem. We complement these results by showing that the min-sum version of the problem cannot be approximated within a factor of $o(\log\Delta)$ on bipartite graphs with uniform costs and on trees with non uniform costs, unless $P = NP$.

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements I want to thank ...

1 Preliminaries

Let $G = (V, E)$ be a connected simple graph. Let $\lambda\colon V \to [k]$ be a function that assigns to each vertex $v \in V$ a set of available interfaces. We wish to find an assignment $\lambda_A\colon V \to [k]$ such that $\lambda_A(v) \subseteq \lambda(v)$ for all $v \in V$ and for every edge $uv \in E$ we have $\lambda_A(u) \cap \lambda_A(v) \neq \emptyset$. We call such an assignment a *covering assignment*. We additionally assume, that a cost function $c\colon [k] \times V \to \mathbb{N}_{\geq 0}$ is given, which denotes the cost of assingining the interface $i \in [k]$ to vertex $v \in V$. The sum cost of a covering assignment is defined as:

$$\texttt{COST}_A = \sum_{v \in V} \sum_{i \in \lambda_A(v)} c(i, v)\,.$$

The goal of the min-sum covering assignment problem is to find a covering assignment λ_A that minimizes $\texttt{COST}_A$. The max cost of a covering assignment is defined as:

$$\texttt{COST}_W = \max_{v \in V} \sum_{i \in \lambda_A(v)} c(i, v)\,.$$

The goal of the min-max covering assignment problem is to find a covering assignment λ_A that minimizes $\texttt{COST}_W$.

© Michał Szyfelbein;
licensed under Creative Commons License CC-BY 4.0
42nd Conference on Very Important Topics (CVIT 2016).
Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:6
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

2 Approximation algorithms for covering multi-interface networks

Below we show an $O(\log n)$ approximation algorithm for both problems. We start with the following ILP formulation of the min-sum covering assignment problem. Let $x_{i,v}$ be a binary variable that is 1 if and only if the interface $i \in [k]$ is assigned to vertex $v \in V$. Then the min-sum covering assignment problem can be formulated as follows:

$$\begin{aligned} \min \quad & \sum_{v \in V} \sum_{i \in \lambda(v)} c(i,v)\, x_{i,v} \\ \text{s.t.} \quad & \sum_{i \in \lambda(u) \cap \lambda(v)} \min\{x_{i,u}, x_{i,v}\} \geq 1 \quad \forall uv \in E \\ & x_{i,v} \in \{0,1\} \quad \forall v \in V, i \in \lambda(v) \end{aligned}$$

Similarly, the min-max covering assignment problem can be formulated as follows:

$$\begin{aligned} \min \quad & M \\ \text{s.t.} \quad & \sum_{i \in \lambda(v)} x_{i,v} c(i,v) \leq M \quad \forall v \in V \\ & \sum_{i \in \lambda(u) \cap \lambda(v)} \min\{x_{i,u}, x_{i,v}\} \geq 1 \quad \forall uv \in E \\ & x_{i,v} \in \{0,1\} \quad \forall v \in V, i \in \lambda(v) \end{aligned}$$

Note that the above formulations are not linear due to the presence of the min function. However, by adding an additional variable $y_{i,uv}$ we can rewrite the constraint $\sum_{i \in \lambda(u) \cap \lambda(v)} \min\{x_{i,u}, x_{i,v}\} \geq 1$ as $\sum_{i \in \lambda(u) \cap \lambda(v)} y_{i,uv} \geq 1$ together with the constraints $y_{i,uv} \leq x_{i,u}$ and $y_{i,uv} \leq x_{i,v}$ for every $i \in \lambda(u) \cap \lambda(v)$ and $uv \in E$. Thus, we can solve the LP relaxation of each of the above formulations in polynomial time.

2.1 Randomized $O(\log m)$-approximation algorithm

Algorithm 1 A relaxation-based $O(\log m)$-approximation algorithm.

1. Solve the LP relaxation and obtain an optimal solution $\{x_{i,v}\}_{i \in \lambda(v), v \in V}$.
2. Pick independent uniformly random thresholds $r_i \in (0,1)$, for each $i \in [k]$, and set $t_i = \frac{r_i}{2 \cdot \ln m}$.
3. Set $\lambda_A(v) = \{i \in \lambda(v) : x_{i,v} \geq t_i\}$, for every $v \in V$.
4. **return** $\{\lambda_A(v)\}_{v \in V}$.

▶ **Theorem 1.** *The above algorithm is an $O(\log m)$-approximation algorithm for both the min-sum and the min-max covering assignment problems which succeeds with high probability.*

Proof. We have the following lemmas:

▶ **Lemma 2.** *The algorithm returns a solution of cost at most $6 \ln m \cdot \mathtt{OPT}$ with high probability.*

Proof. By $X_{v,i}$ denote the indicator random variable that is 1 if $x_{i,v} \geq t_i$ and 0 otherwise. Then we can write $c(\lambda_A(v)) = \sum_{i \in \lambda(v)} X_{v,i} \cdot c(i,v)$. We have: $\mathbb{E}[X_{v,i}] = \Pr[i \in \lambda_A(v)] = 2 \ln m \cdot x_{i,v}$. Thus, $\mathbb{E}[c(\lambda_A(v))] = 2 \ln m \cdot \sum_{i \in \lambda(v)} x_{i,v} \cdot c(i,v)$. By applying the Chernoff bound with $\delta = 2$ we get that

$$\Pr[c(\lambda_A(v)) \geq 3 \cdot \mathbb{E}[c(\lambda_A(v))]] \leq \left(\frac{e^2}{9}\right)^{\mathbb{E}[c(\lambda_A(v))]} \leq \exp\{-\mathbb{E}[c(\lambda_A(v))]\} \leq \frac{1}{m^2}$$

where the last inequality is by using the fact that trivially, $\sum_{i\in\lambda(v)} x_{i,v}\cdot c(i,v) \geq 1$ for all $v \in V$. If otherwise, then v would have no neighbors which contradicts connectivity. By applying the union bound over all n vertices we get that with probability at least $1 - \frac{n}{m^2} \geq 1 - \frac{m+1}{m}$ for every $v \in V$, $c(\lambda_A(v)) \leq 6\ln m \cdot \sum_{i\in\lambda(v)} x_{i,v} \cdot c(i,v)$. Thus, depending on the cost function, with high probability we have $\texttt{COST}_A \leq 6\ln m \cdot \sum_{v\in V}\sum_{i\in\lambda(v)} x_{i,v} \cdot c(i,v) = 6\ln m \cdot \texttt{OPT}$ or $\texttt{COST}_W \leq 6\ln m \cdot \max_{v\in V}\sum_{i\in\lambda(v)} x_{i,v} \cdot c(i,v) = 6\ln m \cdot \texttt{OPT}$. ◀

▶ **Lemma 3.** *With high probability, the above algorithm returns a covering assignment.*

Proof. Consider an edge $uv \in E$. We have:

$$\begin{aligned}
\Pr[\lambda_A(u)\cap\lambda_A(v) = \emptyset] &= \prod_{i\in\lambda(u)\cap\lambda(v)} \Pr[t_i > \min\{x_{i,u}, x_{i,v}\}] \\
&= \prod_{i\in\lambda(u)\cap\lambda(v)} \{1 - 2\ln m \cdot \min\{x_{i,u}, x_{i,v}\}\} \\
&\leq \prod_{i\in\lambda(u)\cap\lambda(v)} \exp\{-2\ln m \cdot \min\{x_{i,u}, x_{i,v}\}\} \\
&= \exp\left\{-2\ln m \cdot \sum_{i\in\lambda(u)\cap\lambda(v)} \min\{x_{i,u}, x_{i,v}\}\right\} \\
&\leq \exp\{-2\ln m\} = \frac{1}{m^2}
\end{aligned}$$

By applying the union bound over all m edges we get that the algorithm returns a covering assignment with probability at least $1 - \frac{1}{m}$. ◀

Combining the above lemmas gives the claim. ◀

3 A k-approximation algorithm for covering multi-interface networks

Algorithm 2 A relaxation-based k-approximation algorithm.

1. Solve the LP relaxation and obtain an optimal solution $\{x_{i,v}\}_{i\in\lambda(v),v\in V}$.
2. Set $\lambda_A(v) = \{i \in \lambda(v) : x_{i,v} \geq \frac{1}{k}\}$, for every $v \in V$.
3. **return** $\{\lambda_A(v)\}_{v\in V}$.

▶ **Theorem 4.** *The above algorithm is a k-approximation algorithm for both the min-sum and the min-max covering assignment problems.*

Proof. Consider an edge $uv \in E$. By averaging, there exists and interface $i \in \lambda(u)\cap\lambda(v)$ such that $\min\{x_{i,u}, x_{i,v}\} \geq \frac{1}{k}$. Thus, $i \in \lambda_A(u)\cap\lambda_A(v)$ and the algorithm returns a covering assignment. Moreover, since $x_{i,v} \geq \frac{1}{k}$ for every $i \in \lambda_A(v)$, we have that $c(\lambda_A(v)) = \sum_{i\in\lambda_A(v)} c(i,v) \leq k \cdot \sum_{i\in\lambda_A(v)} x_{i,v} \cdot c(i,v)$. Thus, depending on the cost function, we have $\texttt{COST}_A \leq k\cdot\sum_{v\in V}\sum_{i\in\lambda(v)} x_{i,v}\cdot c(i,v) = k\cdot\texttt{OPT}$ or $\texttt{COST}_W \leq k\cdot\max_{v\in V}\sum_{i\in\lambda(v)} x_{i,v}\cdot c(i,v) = k\cdot\texttt{OPT}$. ◀

4 $O(\Delta)$-approximation algorithms

4.1 Δ-approximation for the min-sum version

Algorithm 3 A simple Δ-approximation algorithm for the min-sum version.

1 Initialize $\lambda_A(v) = \emptyset$ for every $v \in V$.
2 For every edge $uv \in E$, let $i_{uv} = \operatorname{argmin}_{j \in \lambda(u) \cap \lambda(v)} \{c(j,u) + c(j,v)\}$. Set $\lambda_A(u) = \lambda_A(u) \cup \{i_{uv}\}$ and $\lambda_A(v) = \lambda_A(v) \cup \{i_{uv}\}$.
3 **return** $\{\lambda_A(v)\}_{v \in V}$.

▶ **Theorem 5.** *The above algorithm is a Δ-approximation algorithm for the min-sum covering assignment problem.*

Proof. Consider any optimal solution λ_A^*. For any edge $uv \in E$, let $i_{uv}^* \in \lambda_A^*(u) \cap \lambda_A^*(v)$ be any interface that covers uv in the optimal solution. For every vertex $v \in V$ and edge $uv \in E$ incident to v, we say that v dominates uv, if $c(i_{uv}^*, v) \geq c(i_{uv}^*, u)$ (with ties broken arbitrarily). Let $D(v)$ be the set of edges dominated by v. We have the following inequality:

$$\texttt{OPT} \geq \sum_{v \in V} \max_{uv \in D(v)} \{c(i_{uv}^*, u) + c(i_{uv}^*, v)\}.$$

The cost of the solution built by the procedure is upper bounded by:

$$\begin{aligned}
\sum_{uv \in E} \min_{i \in \lambda(u) \cap \lambda(v)} \{c(i,u) + c(i,v)\} &= \sum_{v \in V} \sum_{uv \in D(v)} \min_{i \in \lambda(u) \cap \lambda(v)} \{c(i,u) + c(i,v)\} \\
&\leq \sum_{v \in V} \sum_{uv \in D(v)} \max_{uv \in D(v)} \{c(i_{uv}^*, u) + c(i_{uv}^*, v)\} \\
&= \sum_{v \in V} \Delta \cdot \max_{uv \in D(v)} \{c(i_{uv}^*, u) + c(i_{uv}^*, v)\} \leq \Delta \cdot \texttt{OPT}
\end{aligned}$$

Since by construction, the solution returned by the procedure is a covering assignment, the claim follows. ◀

4.2 $2 \cdot \Delta$-approximation for the min-max version

We start with the following observation. Let M be any matching in G. Then, we have $\texttt{OPT} \geq \max_{uv \in M} \{\min_{i \in \lambda(u) \cap \lambda(v)} \{\max\{c(i,u), c(i,v)\}\}\}$. This is due to the fact, that for any edge $uv \in M$ there needs to exist a common interface $i \in \lambda_A(u) \cap \lambda_A(v)$ in the optimal solution. Since the above expression represents choosing the (localy) best interface for each edge in the matching, it is upper bounded by the cost of the optimal solution. We use the fact, that for any graph, there always exists a matching incident to every vertex of degree Δ, except at most one. We also note that if G' is a subgraph of G, then $\texttt{OPT}(G') \leq \texttt{OPT}(G)$, since any covering assignment for G is also a covering assignment for G'.

Algorithm 4 $2 \cdot \Delta$-approximation algorithm for the min-max version

1. If $n = 1$ then return $\{\lambda(v) = \emptyset\}$, where v is the only vertex in G.
2. Let M be a matching in G, which is incident to the largest number of vertices of degree Δ. Let $e =$ be an arbitrary edge incident to the remaining vertex of degree Δ, if it exists.
3. For every $H \in G - (M \cup \{e\})$, call the procedure recursively and obtain a covering assignment λ_A for all vertices of H.
4. For every edge $uv \in M \cup \{e\}$, let $i_{uv} = \text{argmin}_{i \in \lambda(u) \cap \lambda(v)} \{\max\{c(i,u), c(i,v)\}\}$. Set $\lambda_A(u) = \lambda_A(u) \cup \{i_{uv}\}$ and $\lambda_A(v) = \lambda_A(v) \cup \{i_{uv}\}$.
5. **return** $\{\lambda_A(v)\}_{v \in V}$.

▶ **Theorem 6.** *The above algorithm is a $2 \cdot \Delta$-approximation algorithm for the min-max covering assignment problem.*

Proof. By construction, the solution returned by the procedure is a covering assignment. We show that its cost is at most $2 \cdot \Delta \cdot \texttt{OPT}$. We claim by induction that this is the case. For $n = 1$, i. e., $\Delta = 1$, the cost is 0 and the claim holds. Assume, that the claim holds for some value of Δ. By definition of M we get that the cost of the solution is upper bounded by:

$$\max_{uv \in M \cup \{e\}} \left\{ \min_{i \in \lambda(u) \cap \lambda(v)} \{\max\{c(i,u), c(i,v)\}\} \right\} + \max_{H \in G - (M \cup \{e\})} \texttt{COST}_W(H)$$
$$\leq 2 \cdot \texttt{OPT}(G) + \max_{H \in G - (M \cup \{e\})} \{2 \cdot (\Delta - 1) \cdot \texttt{OPT}(H)\} \leq 2 \cdot \Delta \cdot \texttt{OPT}(G)$$

where the first inequality is by the fact that M and $\{e\}$ are both matchings and the induction hypothesis. ◀

5 Approximation hardness of covering multi-interface networks

The reduction are from the Set Cover problem, which is defined as follows. We are given a universe U of n elements and a family $\mathcal{S}$ of m subsets of U. The goal is to find a subfamily $\mathcal{S}' \subseteq \mathcal{S}$ of minimum size such that $\bigcup_{S \in \mathcal{S}'} S = U$. It is well known that the Set Cover problem cannot be approximated within a factor of $o(\log n)$, unless $P = NP$.

▶ **Theorem 7.** *The min-sum covering assignment problem cannot be approximated within a factor of $o(\log \Delta)$ on stars, unless $P = NP$.*

Proof. We modify the reduction from the Set Cover problem to the min-sum covering assignment problem given by [cite here] which is as follows: The graph is a star with n leaves, one for each element in U. There are m interfaces, one for each set in $\mathcal{S}$. The center of the star has all m interfaces available, while the leaf corresponding to an element $u \in U$ has only those interfaces available that correspond to sets containing u. The cost of activating any interface at any leaf is 0, while the cost of activating any interface at the center is 1. It is easy to see that there is a one-to-one correspondence between the set covers of U and the covering assignments for the constructed instance. Moreover, the cost of a covering assignment is equal to the size of the corresponding set cover. Thus, if there was an $o(\log \Delta)$-approximation algorithm for the min-sum covering assignment problem on stars, then we would have an $o(\log n)$-approximation algorithm for the Set Cover problem, which contradicts the known hardness result. ◀

▶ **Theorem 8.** *The min-sum covering assignment problem with unit costs cannot be approximated within a factor of $o(\log \Delta)$ even on bipartite graphs, unless $P = NP$.*

Proof. The reduction is similar, however instead of one center vertex, we create n copies of it, with the same set of available interfaces and we connect each leaf to all the copies of the center. The cost of activating any interface at any vertex is 1. Let v be the center representative, such that the cost $c(\lambda_A(v))$ of activating interfaces at v is smallest among all the center representatives. Firstly, we copy the assignment of v to all the center representatives we get a covering assignment, thus obtain a solution with non-increased cost. For every leaf we also leave only one interface activated, which is the one corresponding to some set covering the element represented by this leaf. We therefore have that the cost of the solution is bounded by $cn \leq \texttt{COST} = n + n \cdot c \leq 2 \cdot n \cdot c$. Moreover, by choosing the solution to the set cover instance to be the family of sets corresponding to the interfaces activated at v we get a set cover of size at most c. Since calculating c to within a factor of 2 means calculating the size of the optimal set cover to within a factor of 2, any α-approximation algorithm for the min-sum covering assignment problem would yield an $2 \cdot \alpha$-approximation algorithm for the Set Cover problem, which implies the claim. ◀